

Nama : Radhit Akriandra
NRP : 5025251084
Praktikum 2

Soal 1: Acumalaka

Penjelasan

Kodok memiliki 2 gaya lompatan yaitu,

- **Lompatan super:** Lompatan bernilai sebesar posisi kodok sekarang.
- **Lompatan biasa:** Lompatan bernilai 1 satuan.

Hitunglah jumlah lompatan minimum dari posisi 1 sampai x .

Solusi

Pertama-tama, kita akan loop dari x sampai 1 dengan mempertimbangkan beberapa kondisi, yaitu

- Gunakan lompatan super jika y masih tersisa dan x adalah bilangan genap.
- Gunakan lompatan biasa jika x adalah bilangan ganjil.
- Jika y tak bersisa maka tambahkan dengan sisa akhir dan keluar dari loop.

Agar lompatan minimum, kita akan selalu memprioritaskan lompatan super terlebih dahulu selama y -nya masih ada.

Berikut kode untuk soal di atas.

```
acumalaka.c

#include <stdio.h>
int main(){
    long long x, y;
    scanf("%lld %lld", &x, &y);
    long long ans = 0;
    while(x > 1){
        if(y > 0 && x % 2 == 0){
            x >>= 1;
            y--;
            ans++;
        }else if(y == 0){
            ans += (x - 1);
            break;
        }else{
            x--;
            ans++;
        }
    }
    printf("%lld", ans);
}
```

Soal 2: MR. Number

Penjelasan

Wancoy ingin membeli motor XMAX hanya dengan angka prima. Bantulah wancoy untuk memilih satu atau lebih angka prima yang ada di dompetnya sehingga jumlah uangnya cukup untuk membeli motor.

- Jika uang wancoy cukup, keluarkan `Wah angkanya cocok nih! Angkanya adalah <angka prima>`
- Jika uang wancok tidak cukup, keluarkan `Waduh angkanya kurang nih, minta Bedul dulu deh!`
- Jika tidak ada angka prima di dompet wancoy, keluarkan `Wah angkanya ngga cocok nih!`

Solusi

Kita akan lakukan loop terhadap seluruh uang wancoy secara berurutan dan mengecek apakah terdapat setidaknya 1 angka prima. Jika ada maka tambahkan ke dalam variabel `sum` untuk mengecek apakah jumlah uang wancoy cukup.

Pada primality testing, akan dilakukan sedikit optimasi dengan cara mengeliminasi seluruh bilangan ganjil dan hanya mengecek bilangan prima sampai $\sqrt{num}$ untuk menghindari double checking terhadap faktor `num`.

Berikut kode untuk soal di atas.

`number.c`

```
#include <stdio.h>
int main(){
    long long m, h;
    scanf("%lld %lld", &m, &h);
    long long arr[m];
    for(long long i = 0; i < m; i++){
        scanf("%lld", &arr[i]);
    }
    long long ans[100];
    long long idx = 0, sum = 0, bisa = 0;
    for(long long i = 0; i < m; i++){
        long long num = arr[i];
        int is_prime = 1;
        if(num < 2){
            is_prime = 0;
        }else if(num == 2 || num == 3){
            is_prime = 1;
        }else if(num % 2 == 0 || num % 3 == 0){
            is_prime = 0;
        }else{
            for(long long j = 5; j*j <= num; j+=2){
                if(num % j == 0){
                    is_prime = 0;
                    break;
                }
            }
        }
        if(is_prime){
            sum += num;
            ans[idx] = num;
        }
    }
}
```

```

        idx++;
        bisa = 1;
    }
    if(sum >= h){
        break;
    }
}
if(bisa){
    if(sum >= h){
        printf("Wah angkanya cocok nih! Angkanya adalah ");
        for(int i = 0; i < idx; i++){
            printf("%lld ", ans[i]);
        }
    }else{
        printf("Waduh angkanya kurang nih, minta Bedul dulu deh!\n");
    }
}else{
    printf("Wah angkanya ngga cocok nih!");
}
}

```

Soal 3: Perfectly Balanced

Penjelasan

Diberikan beberapa string palindrom. Tentukan apakah string palindrom tersebut dapat diacak menjadi bentuk palindrom lain yang berbeda.

- Jika dapat diacak, keluarkan `Perfectly balanced, as all things should be.`
- Jika tidak, keluarkan `There was no other way.`

Solusi

Intuisi saya adalah agar sebuah string palindrom dapat diacak menjadi string palindrom yang lain, maka harus ada minimal 2 huruf yang frekuensinya lebih dari 1. Contoh:

- `aba`: Tidak dapat diacak karena hanya ada 1 huruf yang frekuensinya 2.
- `abba`: Dapat diacak menjadi bentuk `baab` karena terdapat 2 huruf yang frekuensinya 2.
- `abacaba`: Dapat diacak menjadi bentuk `aabcbaa` karena terdapat minimal 2 huruf yang frekuensinya 2.

Berdasarkan kondisi di atas, akan dilakukan loop dari indeks 0 sampai indeks $(len + 1)/2$. Untuk menghindari double check, setiap huruf ditandai dalam sebuah array.

Berikut kode untuk soal di atas.

```

perfect.c

#include <stdio.h>
#include <string.h>
int main(){
    int t;
    scanf("%d", &t);
    while(t--){

```

```

char str[100];
int cnt = 0;
int freq[1000];
scanf("%s", str);
int len = strlen(str);
for(int i = 0; i < 1000; i++){
    freq[i] = 0;
}
for(int i = 0; i < len; i++){
    freq[str[i]]++;
}
int udah[1000] = {0};
for(int i = 0; i < (len + 1)/2; i++){
    if(freq[str[i]] >= 2 && !udah[str[i]]){
        cnt++;
        udah[str[i]] = 1;
    }
}
if(cnt >= 2){
    printf("Perfectly balanced, as all things should be\n");
}else{
    printf("There was no other way\n");
}
}

```

Soal 4: Urutan Kudryavka

Penjelasan

Diberikan T jumlah barang yang dicuri oleh Juumonji. Juumonji mencuri barang sesuai dengan huruf depan sesuai urutan alfabet (A, B, C, ...). Bantulah Oreki menentukan pola pencurian Juumonji.

- Jika sesuai urutan, keluarkan `Juumonji mungkin akan mencuri barang dengan huruf <huruf selanjutnya>`.
- Jika tidak sesuai urutan, keluarkan `Apakah Juumonji melakukan kesalahan`.
- Jika terdapat barang yang hilang, keluarkan `Masih ada barang dengan huruf <barang> yang hilang`.

Solusi

Pertama-tama, kita akan menyimpan masing-masing huruf pertama setiap barang dalam sebuah array. Selanjutnya, kita akan mengecek apakah urutan barang sesuai dengan urutan alfabet. Jika terdapat barang yang hilang, kita dapat melakukan looping diantara huruf tersebut dengan menambahkan setiap huruf kedalam sebuah array.

Contoh:

Misalkan terdapat beberapa huruf, yaitu

B, C, D, G, H

Karena D dan G terpisah beberapa huruf, maka kita dapat lakukan loop mulai dari huruf setelah D sampai huruf sebelum G.

Berikut kode untuk soal di atas.

kudryavka.c

```
#include <stdio.h>
int main(){
    int t;
    scanf("%d", &t);
    char first[t];
    for(int i = 0; i < t; i++){
        char str[105];
        scanf(" %[^\n]s", str);
        first[i] = str[0];
        printf("%s telah hilang...\n", str);
    }
    // cek sesuai urutan
    int bisa = 1;
    for(int i = 1; i < t; i++){
        if((int)first[i] <= (int)first[i - 1]){
            bisa = 0;
            break;
        }
    }
    if(!bisa){
        printf("\nApakah Juumonji melakukan kesalahan?");
        return 0;
    }
    // cek barang yang hilang
    int hilang = 0;
    int cnt = 0;
    char ans[100];
    for(int i = 1; i < t; i++){
        int idx = 1;
        if((int)first[i] != (int)(first[i - 1] + 1)){
            hilang = 1;
            while((int)first[i] != (int)(first[i - 1] + idx)){
                ans[cnt] = (char)(first[i - 1] + idx);
                cnt++;
                idx++;
            }
        }
    }
    if(hilang){
        printf("\nMasih ada barang dengan huruf ");
        for(int i = 0; i < cnt; i++){
            printf("%c", ans[i]);
        }
        printf(" yang hilang!");
    }else{
        if(first[t - 1] == 'Z'){
            printf("Juumonji mungkin akan mencuri barang dengan huruf A\n");
        }else{ // barang yang dicuri selanjutnya
            printf("Juumonji mungkin akan mencuri barang dengan huruf %c", (char)(first[t - 1] + 1));
        }
    }
}
```